

Universidade Maurício de Nassau — Maceió

Curso de Engenharia de Software

Controle Financeiro Pessoal

Sistema Web MVP — Documentação Técnica

Trabalho de Avaliação — AV2

Integrantes:

1. João Victor Araujo dos Santos
2. Ruan Welisson Nazário Teles
3. Guilherme Virgílio de Moura Alves
4. Pedro Henrique Mendes de Oliveira
5. Ivano Gabriel Silva Melo
6. Cauã Manoel Nogueira de Aguiar
7. Eduardo dos Santos Silva
8. Pedro Bertonha Sodré
9. Matheus Kayke da Silva Marinho Mariano
10. Lucas da Silva Fernandes

Maceió, 2026

Contents

1	Introdução	3
1.1	Objetivos	3
1.2	Escopo do MVP	3
2	Arquitetura do Sistema	3
2.1	Visão Geral das Camadas	4
2.2	Fluxo de uma Requisição no Back-end	4
3	Stack Tecnológica	5
4	Modelo de Dados	5
4.1	Tabela <code>categorias</code>	5
4.2	Tabela <code>transacoes</code>	5
4.3	Script de Inicialização	6
4.4	Categorias Padrão	6
5	API REST — Endpoints	7
5.1	Payload de Criação/Atualização	7
5.2	Resposta do Endpoint de Resumo	7
6	Implementação do Back-end	8
6.1	Ponto de Entrada — <code>src/index.js</code>	8
6.2	Conexão com o Banco — <code>src/db.js</code>	9
6.3	Camada de Negócio — <code>services/transacoes.service.js</code>	9
6.4	Camada de Dados — <code>repositories/transacoes.repository.js</code>	10
6.5	Lógica de Resumo — <code>services/resumo.service.js</code>	10
7	Implementação do Front-end	12
7.1	Estrutura de Componentes	12
7.2	Serviço de API — <code>src/services/api.js</code>	12
7.3	Gráficos com Recharts	13
8	Containerização com Docker	13
8.1	Configuração do Docker Compose	13
8.2	Portas Expostas	14
9	Estrutura de Pastas	14

10 Guia de Execução	16
10.1 Com Docker (recomendado)	16
10.2 Sem Docker (Desenvolvimento)	16
11 Regras de Negócio	16
12 Padrão de Commits	17
13 Uso de Inteligência Artificial	17
14 Conclusão	18

1 Introdução

Este documento descreve a arquitetura, as decisões de projeto, a implementação e o guia de execução do **Controle Financeiro Pessoal**, um MVP (Minimum Viable Product) desenvolvido como trabalho de avaliação AV2.

O sistema permite que um usuário registre receitas e despesas, organize os lançamentos por categorias e visualize sua situação financeira por meio de um dashboard com indicadores numéricos e gráficos interativos. A entrega demonstra o fluxo completo de uma aplicação web moderna: *front-end* → *back-end* → banco de dados, com cada camada executando em um container Docker independente.

1.1 Objetivos

- Permitir o cadastro, edição e exclusão de transações financeiras (receitas e despesas);
- Organizar lançamentos em categorias pré-definidas;
- Exibir um dashboard com saldo, totais por período e gráficos de evolução;
- Demonstrar a separação em camadas de apresentação, negócio e dados;
- Containerizar toda a aplicação com Docker Compose.

1.2 Escopo do MVP

O MVP prioriza as seguintes funcionalidades:

- CRUD completo de transações (descrição, valor, data, tipo, categoria);
- Categorias pré-definidas para receitas e despesas;
- Filtro de lançamentos por mês/ano;
- Dashboard com saldo, total de receitas e total de despesas;
- Gráfico de rosca — distribuição de despesas por categoria;
- Gráfico de barras — evolução mensal de receitas vs despesas.

Funcionalidades **fora do escopo** desta versão incluem autenticação de múltiplos usuários, integração bancária real (Open Finance), exportação de relatórios e aplicativo mobile.

2 Arquitetura do Sistema

O projeto adota uma arquitetura em **3 camadas**, cada uma executando em um container Docker independente e orquestradas pelo Docker Compose.

2.1 Visão Geral das Camadas

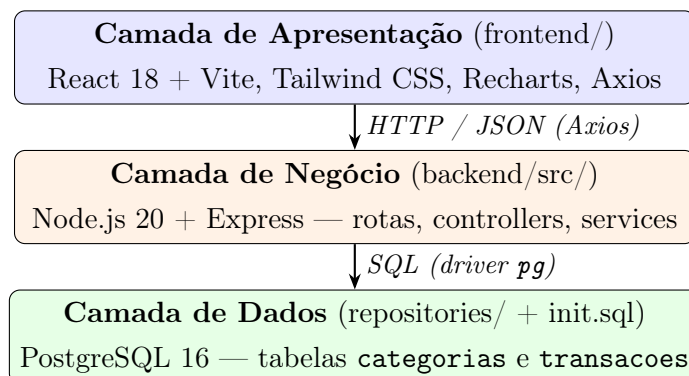


Figure 1: Arquitetura em 3 camadas do sistema

2.2 Fluxo de uma Requisição no Back-end

No back-end, cada chamada HTTP percorre o seguinte caminho:

Rota $\longrightarrow$ Controller $\longrightarrow$ Service $\longrightarrow$ Repository

routes/

Define os endpoints e associa cada verbo HTTP ao controller correspondente.

controllers/

Recebem a requisição, extraem parâmetros e delegam ao service; devolvem a resposta HTTP.

services/

Contêm as **regras de negócio**: validações, cálculo de saldo, agrupamento por categoria, verificação de consistência entre tipo de transação e categoria.

repositories/

Única camada que acessa o banco diretamente, com SQL puro via o driver pg. Não contém regras de negócio.

3 Stack Tecnológica

Table 1: Tecnologias utilizadas por camada

Camada	Tecnologia
Front-end	React 18 (Vite), Tailwind CSS, Recharts, Axios
Back-end	Node.js 20, Express
Banco de dados	PostgreSQL 16
Containerização	Docker + Docker Compose
Versionamento	Git/GitHub (commits semânticos)

4 Modelo de Dados

O banco de dados é composto por duas tabelas principais, criadas pelo script `backend/db/init.sql`.

4.1 Tabela categorias

Table 2: Estrutura da tabela `categorias`

Campo	Tipo	Descrição
<code>id</code>	SERIAL PRIMARY KEY	Identificador único auto-incrementado
<code>nome</code>	VARCHAR(50) NOT NULL	Nome da categoria (ex.: “Alimentação”)
<code>tipo</code>	ENUM('receita','despesa')	Indica se é categoria de receita ou despesa
<code>cor</code>	VARCHAR(7) DEFAULT '#999999'	Cor hexadecimal usada nos gráficos

4.2 Tabela transacoes

Table 3: Estrutura da tabela `transacoes`

Campo	Tipo	Descrição
<code>id</code>	SERIAL PRIMARY KEY	Identificador único
<code>descricao</code>	VARCHAR(255) NOT NULL	Descrição do lançamento
<code>valor</code>	NUMERIC(12,2) CHECK > 0	Valor positivo da transação
<code>data</code>	DATE NOT NULL	Data do lançamento
<code>tipo</code>	ENUM('receita','despesa')	Tipo da transação
<code>categoria_id</code>	INT REFERENCES categorias(id)	Chave estrangeira para a categoria

4.3 Script de Inicialização

O banco é criado e populado automaticamente pelo seguinte script:

```
1 CREATE TYPE tipo_transacao AS ENUM ('receita', 'despesa');
2
3 CREATE TABLE categorias (
4     id SERIAL PRIMARY KEY,
5     nome VARCHAR(50) NOT NULL,
6     tipo tipo_transacao NOT NULL,
7     cor VARCHAR(7) NOT NULL DEFAULT '#999999'
8 );
9
10 CREATE TABLE transacoes (
11     id SERIAL PRIMARY KEY,
12     descricao VARCHAR(255) NOT NULL,
13     valor NUMERIC(12, 2) NOT NULL CHECK (valor > 0),
14     data DATE NOT NULL,
15     tipo tipo_transacao NOT NULL,
16     categoria_id INTEGER REFERENCES categorias(id) ON DELETE SET
17     NULL
18 );
```

Listing 1: Trecho de backend/db/init.sql

4.4 Categorias Padrão

O seed insere 9 categorias iniciais:

Table 4: Categorias pré-definidas no banco

Nome	Tipo	Cor
Salário	receita	#22c55e
Freelance	receita	#06b6d4
Outras Receitas	receita	#84cc16
Alimentação	despesa	#ef4444
Transporte	despesa	#f97316
Moradia	despesa	#a855f7
Lazer	despesa	#ec4899
Saúde	despesa	#3b82f6
Outros	despesa	#6b7280

5 API REST — Endpoints

Table 5: Endpoints disponíveis na API

Método	Rota	Descrição
GET	/api/transacoes?mes=&ano=	Lista transações do período
POST	/api/transacoes	Cria nova transação
PUT	/api/transacoes/:id	Atualiza transação existente
DELETE	/api/transacoes/:id	Remove transação
GET	/api/categorias	Lista categorias disponíveis
GET	/api/resumo?mes=&ano=	Retorna saldo, totais e dados para gráficos
GET	/api/health	Verificação de disponibilidade da API

5.1 Payload de Criação/Atualização

```
1 {  
2   "descricao":    "Supermercado",  
3   "valor":       250.90,  
4   "data":        "2026-06-10",  
5   "tipo":        "despesa",  
6   "categoria_id": 4  
7 }
```

Listing 2: Exemplo de payload para POST/PUT /api/transacoes

5.2 Resposta do Endpoint de Resumo

```
1 {  
2   "mes": 6,  
3   "ano": 2026,  
4   "totalReceitas": 5300.00,  
5   "totalDespesas": 1945.50,  
6   "saldo": 3354.50,  
7   "despesasPorCategoria": [  
8     { "categoria": "Alimentacao", "cor": "#ef4444", "total":  
9       445.50 },  
10    { "categoria": "Moradia", "cor": "#a855f7", "total":  
11      1200.00 }  
12  ],  
13   "evolucaoMensal": [  
14     { "mes": "2026-05", "receita": 4500, "despesa": 1719.90 },  
15     { "mes": "2026-06", "receita": 4500, "despesa": 1719.90 },  
16     { "mes": "2026-07", "receita": 4500, "despesa": 1719.90 },  
17     { "mes": "2026-08", "receita": 4500, "despesa": 1719.90 },  
18     { "mes": "2026-09", "receita": 4500, "despesa": 1719.90 },  
19     { "mes": "2026-10", "receita": 4500, "despesa": 1719.90 },  
20     { "mes": "2026-11", "receita": 4500, "despesa": 1719.90 },  
21     { "mes": "2026-12", "receita": 4500, "despesa": 1719.90 }  
22  ]  
23 }
```



```
13     { "mes": "2026-06", "receita": 5300, "despesa": 1945.50 }
14   ]
15 }
```

Listing 3: Estrutura da resposta de GET /api/resumo

6 Implementação do Back-end

6.1 Ponto de Entrada — src/index.js

O arquivo principal inicializa o Express, configura os middlewares CORS e JSON, registra as rotas e instala um handler centralizado de erros:

```
1 import express from 'express';
2 import cors from 'cors';
3 import dotenv from 'dotenv';
4 import transacoesRoutes from './routes/transacoes.routes.js';
5 import categoriasRoutes from './routes/categorias.routes.js';
6 import resumoRoutes from './routes/resumo.routes.js';
7
8 dotenv.config();
9 const app = express();
10 app.use(cors());
11 app.use(express.json());
12
13 app.use('/api/transacoes', transacoesRoutes);
14 app.use('/api/categorias', categoriasRoutes);
15 app.use('/api/resumo', resumoRoutes);
16 app.get('/api/health', (req, res) => res.json({ status: 'ok' }));
17
18 // Tratamento centralizado de erros
19 app.use((err, req, res, next) => {
20   res.status(err.status || 500).json({
21     erro: err.message || 'Erro interno do servidor',
22     detalhes: err.detalhes,
23   });
24 });
25
26 app.listen(process.env.PORT || 3001);
```

Listing 4: src/index.js — configuração do servidor Express

6.2 Conexão com o Banco — src/db.js

O pool de conexões é configurado via variáveis de ambiente, com valores padrão para desenvolvimento local:

```
1 import { Pool } from 'pg';
2
3 const pool = new Pool({
4   host:      process.env.DB_HOST      || 'localhost',
5   port:      Number(process.env.DB_PORT) || 5432,
6   user:      process.env.DB_USER      || 'financas_user',
7   password:  process.env.DB_PASSWORD  || 'financas_pass',
8   database:  process.env.DB_NAME      || 'financas_db',
9 });
10 export default pool;
```

Listing 5: src/db.js — pool de conexões PostgreSQL

6.3 Camada de Negócio — services/transacoes.service.js

O service valida os dados de entrada antes de qualquer operação no banco:

```
1 async function validarTransacao(dados) {
2   const { descricao, valor, data, tipo, categoria_id } = dados;
3   const erros = [];
4
5   if (!descricao || String(descricao).trim().length === 0)
6     erros.push('A descricao e obrigatoria.');
```

```
23     throw criarErro('Dados invalidos', 400, erros);
24 }
```

Listing 6: Função de validação no service de transações

6.4 Camada de Dados — repositories/transacoes.repository.js

O repository executa SQL puro e não contém lógica de negócio. Abaixo, as queries principais:

```
1  -- Listagem com filtro por mês/ano
2  SELECT t.*, c.nome AS categoria_nome, c.cor AS categoria_cor
3  FROM transacoes t
4  LEFT JOIN categorias c ON c.id = t.categoria_id
5  WHERE EXTRACT(YEAR FROM t.data) = $1
6        AND EXTRACT(MONTH FROM t.data) = $2
7  ORDER BY t.data DESC, t.id DESC;
8
9  -- Totais por tipo e categoria (para o gráfico de pizza)
10 SELECT t.tipo, c.nome AS categoria_nome, c.cor,
11        SUM(t.valor) AS total
12 FROM transacoes t
13 LEFT JOIN categorias c ON c.id = t.categoria_id
14 WHERE EXTRACT(YEAR FROM t.data) = $1
15        AND EXTRACT(MONTH FROM t.data) = $2
16 GROUP BY t.tipo, c.nome, c.cor;
17
18 -- Evolução mensal (para o gráfico de barras)
19 SELECT TO_CHAR(data, 'YYYY-MM') AS mes,
20        tipo, SUM(valor) AS total
21 FROM transacoes
22 GROUP BY mes, tipo
23 ORDER BY mes;
```

Listing 7: Queries do repository de transações

6.5 Lógica de Resumo — services/resumo.service.js

O service de resumo agrega os dados retornados pelo repository e os formata para o front-end:

```
1 export async function gerarResumo({ mes, ano }) {
2   const linhas = await transacoesRepository.resumoPorPeriodo({
3     mes, ano });
```

```
4  let totalReceitas = 0, totalDespesas = 0;
5  const despesasPorCategoria = [];
6
7  for (const linha of linhas) {
8      const total = Number(linha.total);
9      if (linha.tipo === 'receita') {
10         totalReceitas += total;
11     } else {
12         totalDespesas += total;
13         despesasPorCategoria.push({
14             categoria: linha.categoria_nome || 'Sem categoria',
15             cor:        linha.categoria_cor  || '#94a3b8',
16             total,
17         });
18     }
19 }
20
21 return {
22     mes: Number(mes), ano: Number(ano),
23     totalReceitas, totalDespesas,
24     saldo: totalReceitas - totalDespesas,
25     despesasPorCategoria,
26     evolucaoMensal: /* ... agrupamento de evolucaoBruta ... */,
27 };
28 }
```

Listing 8: Cálculo do saldo e agrupamento por categoria

7 Implementação do Front-end

7.1 Estrutura de Componentes

Table 6: Componentes React do front-end

Componente	Responsabilidade
App.jsx	Componente raiz; controla o mês/ano selecionado e busca dados da API
MonthSelector.jsx	Seletor de mês e ano para filtrar o período
Dashboard.jsx	Exibe cards de saldo, total de receitas/despesas e os dois gráficos
TransactionForm.jsx	Formulário modal para criação e edição de transações
TransactionList.jsx	Tabela com listagem e botões de edição/exclusão

7.2 Serviço de API — src/services/api.js

Todas as chamadas HTTP são centralizadas em um módulo que cria uma instância do Axios com a URL base configurada por variável de ambiente:

```
1 import axios from 'axios';
2
3 const api = axios.create({
4   baseURL: import.meta.env.VITE_API_URL ||
5     'http://localhost:3001/api',
6 });
7
8 export const getResumo = (mes, ano) =>
9   api.get('/resumo', { params: { mes, ano } }).then(r =>
10     r.data);
11
12 export const getTransacoes = (mes, ano) =>
13   api.get('/transacoes', { params: { mes, ano } }).then(r =>
14     r.data);
15
16 export const getCategorias = () =>
17   api.get('/categorias').then(r => r.data);
18
19 export const criarTransacao = (dados) =>
20   api.post('/transacoes', dados).then(r => r.data);
21
22 export const atualizarTransacao = (id, dados) =>
```

```

20   api.put('/transacoes/${id}', dados).then(r => r.data);
21
22   export const removerTransacao    = (id) =>
23     api.delete('/transacoes/${id}');

```

Listing 9: src/services/api.js — funções de acesso à API

7.3 Gráficos com Recharts

O componente Dashboard usa a biblioteca Recharts para renderizar dois gráficos:

Gráfico de rosca (PieChart): Exibe a proporção de despesas por categoria usando as cores definidas no banco. Utiliza os componentes PieChart, Pie, Cell e Legend.

Gráfico de barras (BarChart): Compara receitas e despesas mês a mês com barras verdes (receita, #22c55e) e vermelhas (despesa, #ef4444). Utiliza BarChart, Bar, XAxis, YAxis e CartesianGrid.

Os valores são formatados para o padrão monetário brasileiro via Intl.NumberFormat:

```

1   const formatarMoeda = (valor) =>
2     new Intl.NumberFormat('pt-BR', {
3       style: 'currency', currency: 'BRL'
4     }).format(valor || 0);

```

Listing 10: Formatação de moeda no padrão BRL

8 Containerização com Docker

8.1 Configuração do Docker Compose

O arquivo docker-compose.yml define três serviços interdependentes:

```

1   services:
2     db:
3       image: postgres:16-alpine
4       environment:
5         POSTGRES_USER:      financas_user
6         POSTGRES_PASSWORD:  financas_pass
7         POSTGRES_DB:        financas_db
8       volumes:
9         - pgdata:/var/lib/postgresql/data
10
11         ./backend/db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
12     healthcheck:

```

```

12     test: ["CMD-SHELL", "pg_isready -U financas_user -d
13         financas_db"]
14
15 backend:
16     build: ./backend
17     environment:
18         DB_HOST: db
19         PORT:    3001
20     depends_on:
21         db: { condition: service_healthy }
22
23 frontend:
24     build: ./frontend
25     environment:
26         VITE_API_URL: http://localhost:3001/api
27     depends_on: [backend]

```

Listing 11: docker-compose.yml (simplificado)

A dependência `service_healthy` garante que o back-end só inicia **após** o PostgreSQL estar pronto para aceitar conexões, evitando erros de conexão na inicialização.

8.2 Portas Expostas

Table 7: Portas mapeadas pelos containers

Serviço	Imagem	Porta Host	Porta Container
Banco de dados	postgres:16-alpine	5432	5432
Back-end API	Node.js (build)	3001	3001
Front-end	Vite (build)	5173	5173

9 Estrutura de Pastas

```

Trabalho-Aderlan-2.0-main/
+-- docker-compose.yml
+-- escopo_controle_financeiro.md
+-- README.md
+-- backend/
|   +-- Dockerfile
|   +-- package.json
|   +-- db/

```

```
| | \-- init.sql
| \-- src/
|   +-- index.js          # Ponto de entrada da API
|   +-- db.js             # Pool de conexoes PostgreSQL
|   +-- routes/
|     | +-- transacoes.routes.js
|     | +-- categorias.routes.js
|     | \-- resumo.routes.js
|   +-- controllers/
|     | +-- transacoes.controller.js
|     | +-- categorias.controller.js
|     | \-- resumo.controller.js
|   +-- services/
|     | +-- transacoes.service.js
|     | +-- categorias.service.js
|     | \-- resumo.service.js
|   \-- repositories/
|     +-- transacoes.repository.js
|     \-- categorias.repository.js
\-- frontend/
    +-- Dockerfile
    +-- package.json
    +-- index.html
    +-- vite.config.js
    +-- tailwind.config.js
    \-- src/
        +-- main.jsx
        +-- App.jsx
        +-- index.css
        +-- components/
        |   +-- MonthSelector.jsx
        |   +-- Dashboard.jsx
        |   +-- TransactionForm.jsx
        |   \-- TransactionList.jsx
        \-- services/
            \-- api.js
```


10 Guia de Execução

10.1 Com Docker (recomendado)

Pré-requisitos: Docker e Docker Compose instalados.

1. Clone ou extraia o repositório e acesse a pasta raiz do projeto.
2. Execute o comando abaixo para construir as imagens e iniciar os containers:

```
1 docker compose up --build
```

3. Após a inicialização (aguardar o healthcheck do banco):

- Front-end disponível em: <http://localhost:5173>
- API disponível em: <http://localhost:3001/api>

Para encerrar os containers:

```
1 docker compose down
```

Para reinicializar o banco do zero (remove o volume):

```
1 docker compose down -v
2 docker compose up --build
```

10.2 Sem Docker (Desenvolvimento)

Banco de dados: suba um PostgreSQL local e execute o script `backend/db/init.sql`.

Back-end:

```
1 cd backend
2 npm install
3 # Configure: DB_HOST, DB_USER, DB_PASSWORD, DB_NAME via .env
4 npm run dev
```

Front-end:

```
1 cd frontend
2 npm install
3 npm run dev
```

11 Regras de Negócio

1. **Valor positivo:** toda transação deve possuir valor estritamente maior que zero (reforçado tanto no constraint SQL quanto na validação do service).

2. **Tipo coerente com categoria:** ao criar ou editar uma transação, o service verifica se a categoria selecionada é do mesmo tipo (**receita** ou **despesa**) da transação.
3. **Descrição obrigatória:** o campo `descricao` não pode ser vazio ou conter apenas espaços.
4. **Data obrigatória:** o campo `data` deve ser informado.
5. **Categoria existente:** o `categoria_id` deve referenciar uma categoria já cadastrada no banco.
6. **Cálculo do saldo:** $\text{saldo} = \text{totalReceitas} - \text{totalDespesas}$, calculado pelo `resumo.service.js` para o período selecionado.
7. **Exclusão de categoria:** se uma categoria for removida, as transações vinculadas têm `categoria_id` definido como NULL (ON DELETE SET NULL).

12 Padrão de Commits

O projeto adota o padrão *Conventional Commits*:

Table 8: Tipos de commits utilizados

Prefixo	Uso
feat:	Nova funcionalidade
fix:	Correção de bug
docs:	Documentação (README, relatório, escopo)
style:	Ajustes visuais/formatação (sem mudança de lógica)
refactor:	Refatoração sem mudança de comportamento
chore:	Configuração, dependências, Docker

Exemplo: feat: adiciona endpoint de resumo financeiro por período

13 Uso de Inteligência Artificial

Este projeto foi desenvolvido com apoio de IA generativa (Claude). A IA foi utilizada para:

- Estruturar o escopo inicial do projeto (`escopo_controle_financeiro.md`);
- Gerar o esqueleto da arquitetura em 3 camadas do back-end (rotas, controllers, services e repositories);
- Gerar os componentes iniciais do front-end em React + Tailwind (Dashboard, formulário de transações, listagem e seletor de período);

- Criar o `docker-compose.yml` e os Dockerfiles de front-end e back-end.

A equipe é responsável por revisar, ajustar e estender o código gerado, bem como por validar as regras de negócio implementadas. Todo conteúdo produzido com IA foi inspecionado e adaptado pelos integrantes antes de ser incorporado ao projeto.

14 Conclusão

O MVP de Controle Financeiro Pessoal demonstra com sucesso a implementação de uma aplicação web completa em arquitetura de 3 camadas, com frontend React, API Node.js/Express e banco PostgreSQL, todos orquestrados via Docker Compose.

Os principais pontos técnicos evidenciados foram a separação clara de responsabilidades entre rotas, controllers, services e repositories; a consistência do modelo de dados com validações em múltiplas camadas; e a facilidade de implantação proporcionada pela containerização.

Como trabalhos futuros, a aplicação pode ser expandida com autenticação de múltiplos usuários, metas de gastos mensais, exportação de relatórios em PDF e integração com Open Finance para importação automática de extratos bancários.